A Practical Guide to Policy gradient method

TOPIC -- POLICY GRADIENT METHOD

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a \mid s) Q^\pi(s,a)]$$

-- 01 --

Policy gradient The REINFORCE algorithm, introduced by Ronald J. Williams in 1992, was the first policy gradient method. It is based on the identity for the policy gradient ∇ θ J (θ) = E π θ [∑ t = 0 T ∇ θ ln ■ π θ (A t ■ S t) ∑ t = 0 T (γ t R t) | S 0 = s 0] {\displaystyle \nabla _{\theta }J(\theta)=\mathbb {E} _{\pi _{\theta }}\left[\sum _{t=0}^{T}\nabla _{\theta }\ln \pi _{\theta }(A_{t}\mid S_{t})\;\sum _{t=0}^{T}(\gamma ^{t}R_{t}){\Big |}S_{0}=s_{0}\right]} which can be improved via the "causality trick" ∇ θ J (θ) = E π θ [∑ t = 0 T ∇ θ ln ■ π θ (A t ■ S t) ∑ τ = t T (γ τ R τ) | S 0 = s 0] {\displaystyle \nabla _{\theta }J(\theta)=\mathbb {E} _{\pi _{\theta }}\left[\sum _{t=0}^{T}\nabla _{\theta }\ln \pi _{\theta }(A_{t}\mid S_{t})\sum _{\tau =t}^{T}(\gamma ^{\tau }R_{\tau }){\Big |}S_{0}=s_{0}\right]}

-- 02 --

Overview In policy-based RL, the actor is a parameterized policy function π θ {\displaystyle \pi _{\theta }} , where θ {\displaystyle \theta } are the parameters of the actor. The actor takes as argument the state of the environment s {\displaystyle s} and produces a probability distribution π θ (· ■ s) {\displaystyle \pi _{\theta }(\cdot \mid s)} . If the action space is discrete, then ∑ a π θ (a ■ s) = 1 {\displaystyle \sum _{a}\pi _{\theta }(a\mid s)=1} . If the action space is continuous, then ∫ a π θ (a ■ s) d a = 1 {\displaystyle \int _{a}\pi _{\theta }(a\mid s)\mathrm {d} a=1} . The goal of policy optimization is to find some θ {\displaystyle \theta } that maximizes the expected episodic reward J (θ) {\displaystyle J(\theta)} : J (θ) = E π θ [∑ t = 0 T γ t R t | S 0 = s 0] {\displaystyle J(\theta)=\mathbb {E} _{\pi _{\theta }}\left[\sum _{t=0}^{T}\gamma ^{t}R_{t}{\Big |}S_{0}=s_{0}\right]} where γ {\displaystyle \gamma } is the discount factor, R t {\displaystyle R_{t}} is the reward at step t {\displaystyle t} , s 0 {\displaystyle s_{0}} is the starting state, and T {\displaystyle T} is the time-horizon (which can be infinite). The policy gradient is defined as ∇ θ J (θ) {\displaystyle \nabla _{\theta }J(\theta)} . Different policy gradient methods stochastically estimate the

policy gradient in different ways. The goal of any policy gradient method is to iteratively maximize J (θ) {\displaystyle J(\theta)} by gradient ascent. Since the key part of any policy gradient method is the stochastic estimation of the policy gradient, they are also studied under the title of "Monte Carlo gradient estimation".

-- 03 --

γ t (∑ k = 0 n − 1 γ k R t + k + γ n V π θ (S t + n) − V π θ (S t)) {\textstyle \gamma ^{t}\left(\sum _{k=0}^{n-1}\gamma ^{k}R_{t+k}+\gamma ^{n}V^{\pi _{\theta }}(S_{t+n})-V^{\pi _{\theta }}(S_{t})\right)} : n-step TD learning.

-- 04 --

The natural policy gradient method is a variant of the policy gradient method, proposed by Sham Kakade in 2001. Unlike standard policy gradient methods, which depend on the choice of parameters θ {\displaystyle \theta } (making updates coordinate-dependent), the natural policy gradient aims to provide a coordinate-free update, which is geometrically "natural".

-- 05 --

The Group Relative Policy Optimization (GRPO) is a minor variant of PPO that omits the value function estimator V {\displaystyle V} . Instead, for each state s {\displaystyle s} , it samples multiple actions a 1 , … , a G {\displaystyle a_{1},\dots ,a_{G}} from the policy π θ t {\displaystyle \pi _{\theta _{t}}} , then calculate the group-relative advantage A π θ t (s , a j) = r (s , a j) − μ σ {\displaystyle A^{\pi _{\theta _{t}}}(s,a_{j})={\frac {r(s,a_{j})-\mu }{\sigma }}} where μ , σ {\displaystyle \mu ,\sigma } are the mean and standard deviation of r (s , a 1) , … , r (s , a G) {\displaystyle r(s,a_{1}),\dots ,r(s,a_{G})} . That is, it is the standard score of the rewards. Then, it maximizes the PPO objective, averaged over all actions: max θ 1 G ∑ i = 1 G E (s , a 1 , … , a G) ∼ π θ t [{ min (π θ (a i | s) π θ t (a i | s) , 1 + ■) A π θ t (s , a i) if A π θ t (s , a i) > 0 max (π θ (a i | s) π θ t (a i | s) , 1 − ■) A π θ t (s , a i) if A π θ t (s , a i) < 0] {\displaystyle \max _{\theta }{\frac {1}{G}}\sum _{i=1}^{G}\mathbb {E} _{(s,a_{1},\dots ,a_{G})\sim \pi _{\theta _{t}}}\left[{\begin{cases}\min \left({\frac {\pi _{\theta }(a_{i}|s)}{\pi _{\theta _{t}}(a_{i}|s)}},1+\epsilon \right)A^{\pi _{\theta _{t}}}(s,a_{i})&{\text{ if }}A^{\pi _{\theta _{t}}}(s,a_{i})>0\\\max \left({\frac {\pi _{\theta }(a_{i}|s)}{\pi _{\theta _{t}}(a_{i}|s)}},1-\epsilon \right)A^{\pi _{\theta _{t}}}(s,a_{i})&{\text{ if }}A^{\pi _{\theta _{t}}}(s,a_{i})<0\end{cases}}\right]} Intuitively, each policy update step in GRPO makes the policy more likely to respond to each state with an action that performed relatively better than other actions tried at that state, and less likely to respond with one that performed relatively worse. As before, the KL penalty term

can be applied to encourage the trained policy to stay close to a reference policy. GRPO was first proposed in the context of training reasoning language models by researchers at DeepSeek.

-- 06 --

A further improvement is proximal policy optimization (PPO), which avoids even computing F (θ) {\displaystyle F(\theta)} and F (θ) − 1 {\displaystyle F(\theta)^{-1}} via a first-order approximation using clipped probability ratios. Specifically, instead of maximizing the surrogate advantage max θ L (θ , θ t) = E s , a ~ π θ t [π θ (a | s) π θ t (a | s) A π θ t (s , a)] {\displaystyle \max _{\theta }L(\theta ,\theta _{t})=\mathbb {E} _{s,a\sim \pi _{\theta _{t}}}\left[{\frac {\pi _{\theta }(a|s)}{\pi _{\theta _{t}}(a|s)}}A^{\pi _{\theta _{t}}}(s,a)\right]} under a KL divergence constraint, it directly inserts the constraint into the surrogate advantage: max θ E s , a ~ π θ t [{ min (π θ (a | s) π θ t (a | s) , 1 + ■) A π θ t (s , a) if A π θ t (s , a) > 0 max (π θ (a | s) π θ t (a | s) , 1 − ■) A π θ t (s , a) if A π θ t (s , a) < 0] {\displaystyle \max _{\theta }\mathbb {E} _{s,a\sim \pi _{\theta _{t}}}\left[{\begin{cases}\min \left({\frac {\pi _{\theta }(a|s)}{\pi _{\theta _{t}}(a|s)}},1+\epsilon \right)A^{\pi _{\theta _{t}}}(s,a)&{\text{ if }}A^{\pi _{\theta _{t}}}(s,a)>0\\\max \left({\frac {\pi _{\theta }(a|s)}{\pi _{\theta _{t}}(a|s)}},1-\epsilon \right)A^{\pi _{\theta _{t}}}(s,a)&{\text{ if }}A^{\pi _{\theta _{t}}}(s,a)<0\end{cases}}\right]} and PPO maximizes the surrogate advantage by stochastic gradient descent, as usual. In words, gradient-ascending the new surrogate advantage function means that, at some state s , a {\displaystyle s,a} , if the advantage is positive: A π θ t (s , a) > 0 {\displaystyle A^{\pi _{\theta _{t}}}(s,a)>0} , then the gradient should direct θ {\displaystyle \theta } towards the direction that increases the probability of performing action a {\displaystyle a} under the state s {\displaystyle s} . However, as soon as θ {\displaystyle \theta } has changed so much that π θ (a | s) ≥ (1 + ■) π θ t (a | s) {\displaystyle \pi _{\theta }(a|s)\geq (1+\epsilon)\pi _{\theta _{t}}(a|s)} , then the gradient should stop pointing it in that direction. And similarly if A π θ t (s , a) < 0 {\displaystyle A^{\pi _{\theta _{t}}}(s,a)<0} . Thus, PPO avoids pushing the parameter update too hard, and avoids changing the policy too much. To be more precise, to update θ t {\displaystyle \theta _{t}} to θ t + 1 {\displaystyle \theta _{t+1}} requires multiple update steps on the same batch of data. It would initialize θ = θ t {\displaystyle \theta =\theta _{t}} , then repeatedly apply gradient descent (such as the Adam optimizer) to update θ {\displaystyle \theta } until the surrogate advantage has stabilized. It would then assign θ t + 1 {\displaystyle \theta _{t+1}} to θ {\displaystyle \theta } , and do it again. During this inner-loop, the first update to θ {\displaystyle \theta } would not hit the 1 − ■ , 1 + ■ {\displaystyle 1-\epsilon ,1+\epsilon } bounds, but as θ {\displaystyle \theta } is updated further and further away from θ t {\displaystyle \theta _{t}} , it eventually starts hitting the bounds.

For each such bound hit, the corresponding gradient becomes zero, and thus PPO avoid updating θ {\displaystyle \theta } too far away from θ t {\displaystyle \theta _{t}} . This is important, because the surrogate loss assumes that the state-action pair s , a {\displaystyle s,a} is sampled from what the agent would see if the agent runs the policy π θ t {\displaystyle \pi _{\theta _{t}}} , but policy gradient should be on-policy. So, as θ {\displaystyle \theta } changes, the surrogate loss becomes more and more off-policy. This is why keeping θ {\displaystyle \theta } proximal to θ t {\displaystyle \theta _{t}} is necessary. If there is a reference policy π ref {\displaystyle \pi _{\text{ref}}} that the trained policy should not diverge too far from, then additional KL divergence penalty can be added: − β E s , a ∼ π θ t [log ■ (π θ (a | s) π ref (a | s))] {\displaystyle -\beta \mathbb {E} _{s,a\sim \pi _{\theta _{t}}}\left[\log \left({\frac {\pi _{\theta }(a|s)}{\pi _{\text{ref}}(a|s)}}\right)\right]} where β {\displaystyle \beta } adjusts the strength of the penalty. This has been used in training reasoning language models with reinforcement learning from human feedback. The KL divergence penalty term can be estimated with lower variance using the equivalent form (see f-divergence for details): − β E s , a ∼ π θ t [log ■ (π θ (a | s) π ref (a | s)) + π ref (a | s) π θ (a | s) − 1] {\displaystyle -\beta \mathbb {E} _{s,a\sim \pi _{\theta _{t}}}\left[\log \left({\frac {\pi _{\theta }(a|s)}{\pi _{\text{ref}}(a|s)}}\right)+{\frac {\pi _{\text{ref}}(a|s)}{\pi _{\theta }(a|s)}}-1\right]}